React et Supabase : composants, état, hooks et services

I. Composer l'interface avec des composants typés

Un composant est une fonction typée

Un composant reçoit un objet unique, ses **props**, et retourne du JSX. On type le paramètre, pas la fonction : `React.FC` n'apporte plus rien depuis React 18.

```
type SkillBadgeProps = { name: string; level: "beginner" | "confirmed" };

function SkillBadge({ name, level }: SkillBadgeProps) {
  return <span className={`badge badge--${level}`}>{name}</span>;
}

export default function App() {
  return <p>
    <SkillBadge name="TypeScript" level="confirmed" />
    <SkillBadge name="Zod" level="beginner" />
  </p>;
}
```

> *Passez `level="expert"`. Que répond TypeScript, et à quel endroit ?*

React avec TypeScript · React - Passer des props

`children` se déclare explicitement

Le contenu placé entre les balises d'un composant arrive dans la prop `children`. Elle n'existe que si le type la déclare, avec `ReactNode`.

```
import type { ReactNode } from "react";

type PanelProps = { title: string; children: ReactNode };

function Panel({ title, children }: PanelProps) {
  return <section><h2>{title}</h2>{children}</section>;
}

export default function App() {
  return <Panel title="Compétences">
    <p>TypeScript, React, Zod</p>
  </Panel>;
}
```

> *Retirez `children` du type `PanelProps`. Quelle erreur apparaît dans `App` ?*

React - Passer du JSX en children

Composer plutôt que tout afficher

Le parent orchestre, chaque enfant affiche une seule portion de la donnée, avec son propre type. Un changement de maquette ne touche alors qu'un composant.

```
type Project = { title: string; skills: string[] };

function SkillList({ skills }: { skills: string[] }) {
  return <ul>{skills.map((skill) => <li key={skill}>{skill}</li>)}</ul>;
}

function ProjectCard({ project }: { project: Project }) {
  return <article><h3>{project.title}</h3><SkillList skills={project.skills} /></article>;
}

export default function App() {
  return <ProjectCard project={{ title: "Portfolio", skills: ["React", "Zod"] }} />;
}
```

> *Ajoutez `stars: number` au projet et un composant `StarCount`. Quels fichiers changent ?*

React - Votre premier composant

Remonter une action par une prop fonction

Les données descendent en props. Une action remonte par une fonction reçue en prop. Ce composant ne connaît ni `fetch`, ni Supabase.

```
type Article = { id: number; title: string; excerpt: string };

type ArticleCardProps = { article: Article; onSelect: (id: number) => void };

function ArticleCard({ article, onSelect }: ArticleCardProps) {
  return <article>
    <h2>{article.title}</h2><p>{article.excerpt}</p>
    <button type="button" onClick={() => onSelect(article.id)}>Lire l'article</button>
  </article>;
}

export default function App() {
  const article = { id: 1, title: "React", excerpt: "Un article" };
  return <ArticleCard article={article} onSelect={console.log} />;
}
```

> *Remplacez `title` par un nombre dans `App`. Que répond `tsc` ?*

II. Faire vivre l'interface avec l'état

`useState` mémorise une valeur entre deux rendus

Appeler le setter déclenche un nouveau rendu. Le type est inféré depuis la valeur initiale. Annotez-le seulement quand elle ne suffit pas.

```tsx
import { useState } from "react";

export default function App() {
  const [query, setQuery] = useState(""); // inferred: string
  const [selectedId, setSelectedId] = useState<number | null>(null);
  return <main>
    <input value={query} onChange={(event) => setQuery(event.target.value)} />
    <p>Recherche : {query || "(vide)"}</p>
    <button type="button" onClick={() => setSelectedId(1)}>Sélectionner</button>
    <p>Sélection : {selectedId ?? "aucune"}</p>
  </main>;
}
```

> *Retirez `<number | null>`. Pourquoi `setSelectedId(1)` devient-il une erreur ?*

React - L'état, mémoire d'un composant

L'état appartient au parent commun

La liste et le détail partagent l'identifiant sélectionné. Leur parent le possède et le redescend en props.

```
import { useState } from "react";

const titles = [{ id: 1, title: "React" }, { id: 2, title: "Zod" }];

export default function App() {
  const [selectedId, setSelectedId] = useState<number | null>(null);
  const selected = titles.find((item) => item.id === selectedId);
  return <main>
    <ul>{titles.map((item) => <li key={item.id}>
      <button type="button" onClick={() => setSelectedId(item.id)}>{item.title}</button>
    </li>)}</ul>
    {selected && <p>Article sélectionné : {selected.title}</p>}
  </main>;
}
```

> *Si vous cliquez sur deux titres, pourquoi chaque bouton ne garde-t-il pas sa propre sélection ?*

React - Partager l'état entre composants

Quand l'état local ne suffit plus

Le **prop drilling** fait traverser une prop à des composants qui ne l'utilisent pas. Au-delà de deux ou trois niveaux, l'état devient candidat au partage.

SOLUTION	BOILERPLATE	CAS D'USAGE TYPIQUE
Zustand	Minimal	État global simple, choix par défaut du module
Redux Toolkit	Modéré	Grosse équipe, conventions strictes, DevTools
Signals	Minimal	Mises à jour très fréquentes, coût de rendu **mesuré**

Les signals ne sont pas natifs à React : ils passent par une librairie externe.

Règle du module : partager un état seulement quand deux composants éloignés en ont réellement besoin. Un filtre utilisé par une seule page reste local.

> *Dans votre portfolio, citez un état qui doit rester local et un qui pourrait être partagé.*

Zustand · Redux Toolkit

Partager un état avec Zustand

Un store Zustand est un hook. En TypeScript, la double parenthèse `create<T>()(...)` permet l'inférence du reste du store.

```tsx
import { create } from "zustand";

type SearchStore = { query: string; setQuery: (query: string) => void };

export const useSearchStore = create<SearchStore>()((set) => ({
  query: "",
  setQuery: (query) => set({ query }),
}));

export default function App() {
  const query = useSearchStore((state) => state.query);
  const setQuery = useSearchStore((state) => state.setQuery);
  return <input value={query} onChange={(event) => setQuery(event.target.value)} />;
}
```

> *Ajoutez un second composant qui affiche `query`, sans aucune prop. Que se passe-t-il à la saisie ?*

Zustand - Guide TypeScript

Anticiper Next.js : où vit l'état

Sous Next.js App Router, tout composant est un Server Component par défaut : ni `useState`, ni `useEffect`, ni gestionnaire d'événement.

`"use client"` marque une frontière de module. Le fichier et tout ce qu'il importe partent dans le bundle navigateur. Inutile de répéter la directive dans les enfants.

```tsx
// src/components/SearchForm.tsx
"use client";

import { useState } from "react";

export function SearchForm() {
  const [query, setQuery] = useState("");
  return <input value={query} onChange={(event) => setQuery(event.target.value)} />;
}
```

Règle : la directive va sur le composant le plus bas de l'arbre qui a réellement besoin d'état.

> *Parmi `SkillBadge`, `Panel` et `SearchForm`, lesquels resteraient côté serveur ?*

Next.js - Server and Client Components

III. Charger des données avec un effet et un hook

Un effet synchronise le composant avec l'extérieur

Le rendu doit rester pur. Un timer, un appel réseau ou un abonnement vont dans `useEffect`, qui s'exécute après l'affichage. La fonction retournée nettoie.

```
import { useEffect, useState } from "react";

export default function App() {
  const [seconds, setSeconds] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setSeconds((value) => value + 1), 1000);
    return () => clearInterval(id);
  }, []);
  return <p>Page ouverte depuis {seconds} s</p>;
}
```

`[]` signifie « aucune dépendance » : l'effet s'exécute au montage et se nettoie au démontage.

> *Remplacez par `setSeconds(seconds + 1)`. Pourquoi le compteur reste-t-il bloqué à 1 ?*

React - Synchroniser avec des Effects

Un premier `fetch` dans un effet

Le nettoyage lève un drapeau `ignore`. Une réponse arrivée après le démontage n'écrit plus dans l'état.

```
import { useEffect, useState } from "react";

const url = "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles?select=title";
const headers = { apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV" };

export default function App() {
  const [titles, setTitles] = useState<string[]>([]);
  useEffect(() => {
    let ignore = false;
    fetch(url, { headers })
      .then((response) => response.json())
      .then((rows: { title: string }[]) => { if (!ignore) setTitles(rows.map((row) => row.title)); });
    return () => { ignore = true; };
  }, []);
  return <ul>{titles.map((title) => <li key={title}>{title}</li>)}</ul>;
}
```

Deux défauts volontaires : aucune erreur gérée, et `rows` est annoté sans être validé.

> *Changez la clé. Que voit l'utilisateur ? Que devrait-il voir ?*

Modéliser chargement, erreur et succès

Trois booléens autorisent `loading` et `error` vrais en même temps. Une union discriminée rend cet état impossible à écrire.

```
type State =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; titles: string[] };

function TitlesView({ state }: { state: State }) {
  if (state.status === "loading") return <p role="status">Chargement...</p>;
  if (state.status === "error") return <p role="alert">{state.message}</p>;
  return <ul>{state.titles.map((title) => <li key={title}>{title}</li>)}</ul>;
}

export default function App() {
  return <TitlesView state={{ status: "error", message: "HTTP 401" }} />;
}
```

> *Lisez `state.titles` dans la branche `error`. Que répond TypeScript ?*

React - Rendu conditionnel · TypeScript - Narrowing

Extraire un hook personnalisé

Un hook regroupe état et effet sous un nom. Le drapeau garde **chaque** `setState`, y compris celui du `catch`.

```
// url, headers, State and TitlesView come from the previous slides
function useTitles(): State {
  const [state, setState] = useState<State>({ status: "loading" });
  useEffect(() => {
    let ignore = false;
    fetch(url, { headers })
      .then((response) => {
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        return response.json() as Promise<{ title: string }[]>;
      })
      .then((rows) => { if (!ignore) setState({ status: "success", titles: rows.map((row) => row.title) }); })
      .catch((error: unknown) => { if (!ignore) setState({ status: "error",
        message: error instanceof Error ? error.message : "Unknown error" }); });
    return () => { ignore = true; };
  }, []);
  return state;
}
```

> *Appelez `useTitles()` deux fois dans `App`. Combien de requêtes ? L'état est-il partagé ?*

React - Hooks personnalisés

Ne pas généraliser trop tôt

Une abstraction se justifie à partir de deux cas d'usage concrets et différents. Avant, le code direct reste plus lisible.

```
// Over-engineering for a single resource type
function createResourceHook<T>(config: { endpoint: string; parse: (raw: unknown) => T }) {
  /* ... */
}

// Enough while the app only reads articles
function useArticles(query: string, page: number) {
  /* ... */
}
```

Règle : généraliser après avoir observé la duplication, jamais avant.

En production, TanStack Query ou SWR ajoutent cache, déduplication et nouvelles tentatives. Le module écrit le hook à la main pour comprendre ce qu'ils automatisent.

> 🛠️ ***TP (~180 min)*** *: construire un explorateur d'API publique en React et TypeScript, avec composants imbriqués et typés, états de chargement et d'erreur explicites, dans un dépôt Git aux commits soignés.*

Ouvrir le TP · Correction

IV. Brancher le portfolio sur Supabase

Du JSON à l'interface

Le portfolio reçoit du JSON de l'API REST Supabase. TypeScript ne contrôle pas ce JSON à l'exécution.

```
Supabase REST → service HTTP + Zod → hook React → composants → interface
```

- Le **service** construit l'URL, appelle l'API et valide la réponse.
- Le **hook** suit la requête pendant la vie du composant.
- Le **composant** reçoit des données validées et affiche une interface.

Le hook `useTitles` faisait les trois à la fois. Cette section les sépare.

> *Où traiter un statut HTTP 500 ? Où afficher « Aucun article » ?*

Supabase - Data API · Zod - Bases

Rechercher avec `ilike`

PostgREST filtre une colonne avec `colonne=opérateur.valeur`. Dans `ilike`, `*` remplace `%` et la casse est ignorée.

```
function buildArticlesUrl(query: string, page: number, size = 6): URL {
  const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles");
  url.searchParams.set("select", "id,title,slug");
  url.searchParams.set("order", "created_at.desc,id.desc");
  url.searchParams.set("limit", String(size));
  url.searchParams.set("offset", String((page - 1) * size));
  const term = query.trim();
  if (term) url.searchParams.set("title", `ilike.*${term}*`);
  return url;
}

const url = buildArticlesUrl("React & Zod", 2);
console.log(url.searchParams.get("title"));
console.log(url.search);
```

`URLSearchParams` encode `&` et les espaces. Une concaténation brute couperait la recherche en deux paramètres.

> *Appelez `fetch(buildArticlesUrl("TYPESCRIPT", 1), { headers: { apikey } })`. Combien d'articles ?*

Filtres PostgREST · MDN - URLSearchParams

Compter les lignes avec `Content-Range`

La réponse ne contient que la page demandée. L'en-tête `Prefer: count=exact` ajoute le total dans `Content-Range`.

```
const url = new URL("https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles");
url.searchParams.set("select", "id,title");
url.searchParams.set("limit", "6");
url.searchParams.set("offset", "0");

const response = await fetch(url, {
  headers: {
    apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV",
    Prefer: "count=exact",
  },
});

const contentRange = response.headers.get("Content-Range");
console.log(response.status, contentRange);
console.log(Number(contentRange?.split("/")[1]));
```

`0-0/1` se lit « lignes 0 à 0, sur 1 au total ». Le total tient compte de RLS et des filtres.

> *Passez `offset` à `60`. Quel statut obtenez-vous et que dit le corps de la réponse ?*

Supabase - Pagination et comptage · PostgREST - Exact count

Lire la configuration une seule fois

Le cours précédent a placé l'URL et la clé publishable dans `.env.local`. Ce module les valide au démarrage.

```ts
// src/services/supabase.ts
import { z } from "zod";

const envSchema = z.object({
  VITE_SUPABASE_URL: z.url(),
  VITE_SUPABASE_PUBLISHABLE_KEY: z.string().startsWith("sb_publishable_"),
});

const env = envSchema.parse(import.meta.env);

export const apiHeaders = { apikey: env.VITE_SUPABASE_PUBLISHABLE_KEY };

export function restUrl(table: string): URL {
  return new URL(`/rest/v1/${table}`, env.VITE_SUPABASE_URL);
}
```

`startsWith` refuse une clé `sb_secret_...` collée par erreur. Elle finirait sinon dans le bundle public.

> *Retirez la clé de `.env.local`, relancez `npm run dev`. Quel message apparaît, et où ?*

Vite - Variables d'environnement · Supabase - Clés API

Écrire le contrat du service

Le schéma décrit exactement le `select`. `readTotal` refuse une réponse sans total exploitable.

```
// src/types/article.ts
import { z } from "zod";

export const articlePreviewSchema = z.object({
  id: z.number().int(),
  title: z.string(),
  slug: z.string(),
  excerpt: z.string(),
  cover_url: z.string().nullable(),
  cover_alt: z.string().nullable(),
});
export const articlePreviewListSchema = z.array(
  articlePreviewSchema,
);
export type ArticlePreview = z.infer<typeof articlePreviewSchema>;
export type ArticlePage = { articles: ArticlePreview[]; total: number };

// src/services/pagination.ts
export const PAGE_SIZE = 6;
export function readTotal(contentRange: string | null): number {
  const total = Number(contentRange?.split("/")[1]);
  if (!Number.isInteger(total) || total < 0) throw new Error("Missing total count");
  return total;
}
```

Zod - Tableaux · Zod - Nullable

Valider à la frontière du service

Le service refuse une erreur HTTP et un JSON incompatible. Le composant ne voit que des `ArticlePreview`.

```ts
// src/services/articles.ts
import { articlePreviewListSchema, type ArticlePage } from "../types/article";
import { PAGE_SIZE, readTotal } from "./pagination";
import { apiHeaders, restUrl } from "./supabase";
export function buildArticlesUrl(query: string, page: number): URL {
  const url = restUrl("articles");
  url.searchParams.set("select", "id,title,slug,excerpt,cover_url,cover_alt");
  url.searchParams.set("order", "created_at.desc,id.desc");
  url.searchParams.set("limit", String(PAGE_SIZE));
  url.searchParams.set("offset", String((page - 1) * PAGE_SIZE));
  const term = query.trim();
  if (term) url.searchParams.set("title", `ilike.*${term}*`);
  return url;
}

export async function fetchArticles(query: string, page: number, signal?: AbortSignal): Promise<ArticlePage> {
  const headers = { ...apiHeaders, Prefer: "count=exact" };
  const response = await fetch(buildArticlesUrl(query, page), { headers, signal });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const result = articlePreviewListSchema.safeParse(await response.json());
  if (!result.success) throw new Error("Invalid articles response");
  return { articles: result.data, total: readTotal(response.headers.get("Content-Range")) };
}
```

Supabase - API REST · Zod - safeParse

Tester le contrat sans React

Le contrat est un module pur. Vitest le teste sans navigateur, sans réseau et sans `.env.local`.

```ts
// src/types/article.test.ts
import { describe, expect, it } from "vitest";
import { readTotal } from "../services/pagination";
import { articlePreviewListSchema } from "./article";

const row = { id: 1, title: "TS", slug: "ts", excerpt: "Guide", cover_url: null, cover_alt: null };

describe("articles contract", () => {
  it("accepts a page and an empty page", () => {
    expect(articlePreviewListSchema.parse([row])).toHaveLength(1);
    expect(articlePreviewListSchema.parse([])).toEqual([]);
  });
  it("rejects a numeric title", () => {
    expect(articlePreviewListSchema.safeParse([{ ...row, title: 42 }]).success).toBe(false);
  });
  it("reads the total from Content-Range", () => {
    expect(readTotal("0-5/13")).toBe(13);
    expect(readTotal("*/0")).toBe(0);
    expect(() => readTotal(null)).toThrow();
  });
});
```

> *Ajoutez un test : une ligne sans `cover_url` doit être refusée. Pourquoi n'est-ce pas `null` ?*

Vitest - Premiers pas

Brancher le hook au service validé

Le drapeau `ignore` devient un `AbortController` : la requête obsolète est annulée, pas seulement ignorée. Le résultat mémorise sa clé de requête.

```
// src/hooks/useArticles.ts
import { useEffect, useState } from "react";
import type { ArticlePage } from "../types/article";
import { fetchArticles } from "../services/articles";
type Settled = { status: "error"; message: string } | { status: "success"; data: ArticlePage };
export type ArticlesState = { status: "loading" } | Settled;
export function useArticles(query: string, page: number): ArticlesState {
  const key = `${query}|${page}`;
  const [result, setResult] = useState<{ key: string; state: Settled } | null>(null);
  useEffect(() => {
    const controller = new AbortController();
    fetchArticles(query, page, controller.signal)
      .then((data): Settled => ({ status: "success", data }))
      .catch((error: unknown): Settled =>
        ({ status: "error", message: error instanceof Error ? error.message : "Unknown error" }))
      .then((state) => {
        if (!controller.signal.aborted) setResult({ key, state });
      });
    return () => controller.abort();
  }, [key, query, page]);
  return result?.key === key ? result.state : { status: "loading" };
}
```

React - Récupérer des données dans un Effect

Afficher quatre résultats distincts

Le composant décide du rendu. Une liste vide n'est pas une erreur réseau.

```
// src/app/App.tsx
import { useArticles } from "../hooks/useArticles";

export default function App() {
  const state = useArticles("", 1);
  if (state.status === "loading") return <p role="status">Chargement...</p>;
  if (state.status === "error") return <p role="alert">{state.message}</p>;
  if (state.data.articles.length === 0) return <p>Aucun article publié.</p>;
  return <main>
    <h1>Articles ({state.data.total})</h1>
    <ul>{state.data.articles.map((article) =>
      <li key={article.id}>{article.title}</li>)}
    </ul>
  </main>;
}
```

> *Testez une clé `sb_publishable_` erronée, une recherche sans résultat, puis la liste publiée.*

React - Rendu conditionnel · WCAG - Messages d'état

Réinitialiser la page après recherche

Le parent garde `query` et `page`, car recherche et pagination utilisent les deux valeurs.

```
import { useState } from "react";

const PAGE_SIZE = 6;
const total = 13;

export default function App() {
  const [query, setQuery] = useState("");
  const [page, setPage] = useState(1);
  const hasNext = page * PAGE_SIZE < total;
  function search(nextQuery: string) { setQuery(nextQuery); setPage(1); }
  return <main>
    <form onSubmit={(event) => { event.preventDefault();
      search(new FormData(event.currentTarget).get("query")?.toString() ?? ""); }}>
      <label>Recherche <input name="query" defaultValue={query} /></label>
      <button type="submit">Rechercher</button>
    </form>
    <p>Recherche : {query}, page : {page}</p>
    <button type="button" disabled={!hasNext} onClick={() => setPage(page + 1)}>Page suivante</button>
  </main>;
}
```

> *Passez à la page 3, puis cherchez « Zod ». Quelle page s'affiche ? Et sans `setPage(1)` ?*

React - Partager l'état

Choisir la bonne couche

Classez chaque modification avant de coder.

1. Le titre de l'article est un nombre dans le JSON : schéma Zod du contrat.
2. Une requête ancienne remplace la recherche récente : clé et nettoyage du hook.
3. Un article sans couverture doit rester lisible : composant de carte.
4. Le filtre doit revenir à la page 1 : parent qui possède `query` et `page`.
5. Un numéro de page au-delà de la dernière renvoie `416` : bouton borné par `total` dans le parent.

> *Pour chaque ligne, indiquez le fichier à ouvrir dans le TP.*

React - Garder les composants purs · Zod - Bases

TP : connecter le portfolio React à Supabase

> 🛠️ ***TP (~120 min)*** *: créer un nouveau dépôt Git pour un portfolio React qui lit ses articles dans Supabase.*

Palier 1 obligatoire - Créez une application React/Vite neuve. Ajoutez `supabase.ts`, `types/article.ts` et `articles.ts`, puis affichez la liste avec recherche et quatre états. Résultat attendu : la liste s'affiche sans assertion `as` sur le JSON. `npm test` et `npm run build` réussissent.

Palier 2 - Dérivez le schéma de l'aperçu avec `.pick()`, ouvrez le détail par `slug`, paginez avec `Content-Range`. Résultat attendu : pages 1 et 2 distinctes, aucune requête `416`, du HTML dans `content` s'affiche comme du texte.

Palier 3 bonus - Projets et compétences liées, page « À propos », cache, carte client et serveur pour Next.js. Résultat attendu : compétences typées sans `as`, page déjà visitée réaffichée sans nouvelle requête.

Ouvrir le TP · Correction

Documentation React · API REST Supabase